

Optimisation — Portfolio & Quantitative Finance

Objective Functions, Constraints, and the Mathematics of the Best Available Decision

From Efficient Frontiers to Risk Parity: Turning Signals and Risk Estimates into Portfolios

A Level 2 module in the Quantitative Finance Curriculum — revisited at Level 5

Covers: objective functions · constraints · unconstrained/constrained optimisation · Lagrange multipliers · convex optimisation · quadratic programming · mean-variance optimisation · efficient frontier · minimum-variance portfolios · maximum-Sharpe portfolios · risk parity · Equal Risk Contribution (ERC) · transaction costs · turnover constraints · regularisation · robust optimisation

Table of Contents

How to Use This Guide

What You Need to Know Before Starting

Understand Before Proceeding

Why It Matters

How This Connects to the Rest of the Curriculum

Glossary of Recurring Terms

Part 1 — Formulating an Optimisation Problem

1.1 Objective Functions and Decision Variables

1.2 Constraints: Equality, Inequality, and Box

1.3 The Feasible Region and the Optimal Solution

Part 2 — Unconstrained and Constrained Optimisation

2.1 Unconstrained Optimisation and Optimality Conditions

2.2 Constrained Optimisation and Lagrange Multipliers

2.3 The Karush-Kuhn-Tucker (KKT) Conditions (awareness)

Part 3 — Convex Optimisation and Quadratic Programming

3.1 Convex Sets and Convex Functions

3.2 Why Convexity Is the Property That Makes Optimisation Tractable

3.3 Quadratic Programming

Part 4 — Mean-Variance Optimisation and the Efficient Frontier

4.1 The Markowitz Mean-Variance Framework

4.2 The Efficient Frontier

4.3 Minimum-Variance and Maximum-Sharpe Portfolios

Part 5 — Risk-Based Portfolio Construction

5.1 Risk Parity

5.2 Equal Risk Contribution (ERC)

Part 6 — Making Optimisation Realistic: Costs, Stability, and Robustness

6.1 Transaction Costs and Turnover Constraints

6.2 Regularisation and Shrinkage

6.3 Robust Optimisation (awareness)

Practical Exercise

Common Beginner Mistakes

Advanced Extensions

Professional Applications

Where to Go Next

How to Use This Guide

This document is a companion to Optimisation — Portfolio & Quantitative Finance, the module that sits between a good forecast and a usable portfolio. Every other module in this curriculum tells you something about the world — expected returns, risk, correlations, signals. This module is about what to do with all of that: how to turn a set of estimates and a set of real-world limits into a single, implementable set of numbers — portfolio weights, trade sizes, or hedge ratios.

The module is self-contained: every concept is introduced from first principles, paired with a worked numerical example, and illustrated with a chart or diagram wherever a picture makes the idea land faster than prose. Read it start to finish if optimisation is new to you, or jump straight to the section you need — but do not skip Understand Before Proceeding below.

What You Need to Know Before Starting

- Calculus — partial derivatives, gradients, and the idea of a stationary point where a derivative equals zero.
- Linear Algebra — vectors, matrices, matrix-vector products, and (ideally) what a positive-definite matrix is, since covariance matrices are one.
- Probability & Statistics — expected value, variance, and covariance, since these are the raw inputs to most portfolio-optimisation problems.
- Returns & Portfolio Mathematics — how portfolio return and portfolio variance are built from individual asset weights, returns, and the covariance matrix.
- Risk Metrics — volatility, Value-at-Risk, and related measures, since many optimisation objectives are risk measures being minimised or constrained.

Understand Before Proceeding

Prerequisite gate — An optimiser does not create an investment edge.

It finds the mathematically optimal solution implied by the objective, the inputs, and the constraints you provide — nothing more. If the inputs are wrong, the “optimal” output will be confidently, precisely wrong. Poorly estimated expected returns, covariance matrices, or risk parameters routinely produce portfolios that are highly unstable and economically meaningless, even when the solver reports that it found the global optimum without error. Everything in this module is deliberately at awareness depth for the harder stability and robustness topics (shrinkage theory, robust and distributionally-robust optimisation, dynamic/stochastic optimisation) — enough to reason correctly about why naive optimisation fails in practice, but not enough to safely design a production portfolio-construction system. That requires the Depth 3+ treatment of this same material.

Why It Matters

Optimisation is the hinge on which the entire quantitative investment process turns. Research produces forecasts and risk estimates; optimisation converts them into a portfolio; trading implements the portfolio. A brilliant forecasting model attached to a naive or unstable optimiser can still produce a poor, high-turnover, concentrated portfolio that never realises the forecast's theoretical edge. Conversely, disciplined optimisation — with sensible constraints, regularisation, and cost-awareness — can meaningfully improve outcomes even from mediocre forecasts, simply by allocating capital and risk more efficiently than an ad hoc or equal-weighted approach would.

The practical stakes are large and easy to underestimate. A mean-variance optimiser handed noisy sample estimates of expected returns will happily produce extreme, highly concentrated “optimal” weights — large long positions in a few assets whose historical returns looked good purely by chance, and correspondingly large short positions elsewhere to “hedge” against noise in the estimated covariances. This is not a bug in the mathematics; it is the mathematics working exactly as specified on unreliable inputs. Recognising this — and knowing the standard remedies, from shrinkage to explicit constraints to robust formulations — is the difference between an optimiser that helps and one that quietly destroys value.

How This Connects to the Rest of the Curriculum

- **Research** — Optimisation is used to translate alpha signals into portfolios, construct efficient portfolios, allocate risk across assets, and incorporate transaction costs and portfolio constraints directly into a backtest rather than bolting them on afterward.
- **Development** — Optimisation routines should be implemented as reusable, deterministic, and well-tested components. Solvers such as quadratic-programming and general convex-optimisation libraries are commonly integrated into portfolio-construction and execution infrastructure, and their numerical behaviour (tolerances, warm-starts, infeasibility handling) needs the same engineering discipline as any other production code.
- **Trading** — Optimisation determines position sizing, portfolio weights, hedging ratios, execution schedules, and capital allocation. In market-neutral and statistical-arbitrage strategies specifically, it determines how exposure is distributed across the book while satisfying neutrality, leverage, and risk constraints simultaneously.

Glossary of Recurring Terms

Objective function — The single quantity an optimisation problem seeks to maximise or minimise — e.g., expected return, portfolio variance, or a risk-adjusted combination of the two.

Decision variable — A quantity the optimiser is free to choose, such as a portfolio weight, a trade size, or a model parameter.

Constraint — A restriction the chosen decision variables must satisfy — an equation (equality constraint) or inequality that defines what counts as an allowed solution.

Feasible region — The set of all decision-variable values that satisfy every constraint simultaneously; the optimiser searches only within this set.

Convexity — A property of a set or function guaranteeing, loosely, that there are no “traps” — any local optimum is automatically the global optimum.

Lagrange multiplier — A variable introduced to convert a constrained optimisation problem into an unconstrained one, whose value measures how much the optimal objective would improve if the corresponding constraint were relaxed slightly.

Quadratic programming (QP) — Optimisation of a quadratic objective function subject to linear constraints — the workhorse formulation of mean-variance portfolio optimisation.

Efficient frontier — The set of portfolios that deliver the highest possible expected return for each level of risk (equivalently, the lowest possible risk for each level of expected return).

Minimum-variance portfolio — The single portfolio on the efficient frontier with the lowest possible variance, regardless of expected return.

Maximum-Sharpe (tangency) portfolio — The portfolio on the efficient frontier that maximises the Sharpe ratio — equivalently, the point where a line from the risk-free rate is tangent to the frontier.

Risk parity — An allocation approach that sizes positions so that each asset (or asset class) contributes an equal amount of capital, scaled inversely to its volatility, rather than equal amounts of capital.

Equal Risk Contribution (ERC) — A generalisation of risk parity that solves explicitly for weights such that every asset contributes an equal share of total portfolio risk, accounting for correlations as well as volatilities.

Turnover — The amount of trading (buying and selling) required to move a portfolio from its current weights to a new target, usually expressed as a fraction of assets under management.

Regularisation / shrinkage — A family of techniques that deliberately bias an estimate (e.g., a covariance matrix) toward a simpler, more stable target, trading a small amount of bias for a large reduction in estimation-error-driven instability.

Robust optimisation — An optimisation approach that seeks a solution which performs well across a specified range of possible input values, rather than one that is optimal only for a single point estimate.

Part 1 — Formulating an Optimisation Problem

Every optimisation problem in quantitative finance — no matter how sophisticated — reduces to the same three ingredients: a decision the optimiser is free to make, a number it is trying to make as large or as small as possible, and a set of rules the decision must obey. This part covers all three, and the vocabulary that describes the space in which the optimiser searches.

1.1 Objective Functions and Decision Variables

The decision variables are whatever the optimiser is free to choose: portfolio weights $w_1, \dots, w_n$ across n assets, the size of a single trade, or the parameters of a trading model. The objective function is the single number, computed from those decision variables, that the optimiser is instructed to maximise or minimise. In portfolio construction the objective is often expected portfolio return, portfolio variance, a Sharpe ratio, or some explicit combination of return and risk.

Generic optimisation problem: maximise (or minimise) $f(w)$ subject to $w \in \text{Feasible region}$

A concrete and central example is mean-variance utility, which combines expected return and risk into a single scalar objective using a risk-aversion parameter λ :

$$U(w) = w^T \mu - (\lambda/2) \cdot w^T \Sigma w$$

Here w is the vector of portfolio weights, μ is the vector of expected asset returns, Σ is the covariance matrix of asset returns, and λ controls how heavily the investor penalises variance relative to expected return. A larger λ produces a more conservative, lower-variance portfolio; a smaller λ produces a more aggressive, higher-return-seeking one. Every mean-variance optimisation in this module is, at its core, a variant of maximising this single expression.

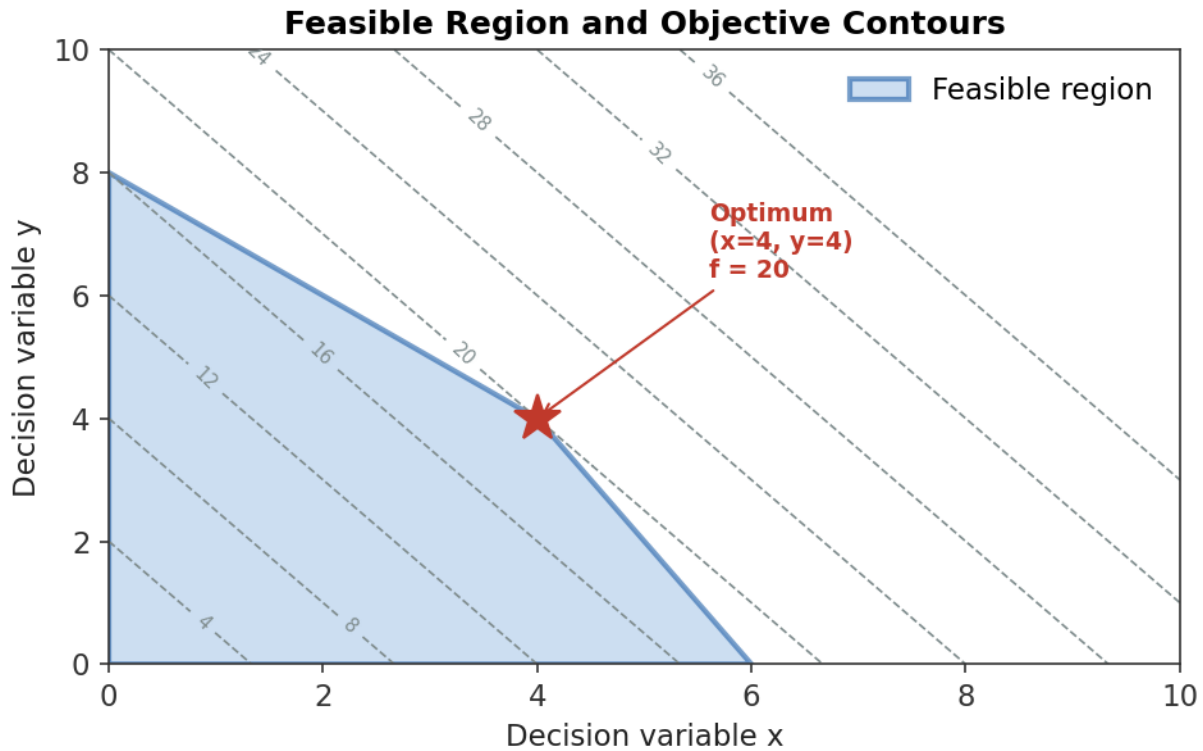


Figure 1.1 — A two-variable optimisation problem: shaded region is feasible, dashed lines are contours of equal objective value, and the star marks the feasible point achieving the highest objective value.

Worked example — A two-asset objective.

An investor holds two assets with expected returns $\mu = (8\%, 5\%)$ and wants to maximise $U(w) = w^T \mu - (\lambda/2) \cdot w^T \Sigma w$ with $\lambda = 4$, given $\sigma_1 = 20\%$, $\sigma_2 = 12\%$, and correlation $\rho = 0.3$ (so $\Sigma_{12} = 0.20 \times 0.12 \times 0.3 = 0.0072$). For weights $w = (0.6, 0.4)$: expected return = $0.6 \times 8\% + 0.4 \times 5\% = 6.8\%$. Portfolio variance = $0.6^2 \times 0.20^2 + 0.4^2 \times 0.12^2 + 2 \times 0.6 \times 0.4 \times 0.0072 = 0.0144 + 0.0023 + 0.0035 = 0.0202$. Utility = $0.068 - 2 \times 0.0202 = 0.068 - 0.0404 = 0.0276$. The optimiser's job is to search over every feasible w and find the one that makes this final number as large as possible.

1.2 Constraints: Equality, Inequality, and Box

Constraints define what counts as an allowed answer. An equality constraint forces a quantity to take an exact value — most commonly, that portfolio weights sum to one (fully invested, no cash drag): $\sum w_i = 1$. An inequality constraint restricts a quantity to a range without pinning it to a single value — for example, no single position may exceed 10% of the portfolio: $w_i \leq 0.10$ for every i . A box constraint is simply a pair of inequality constraints bounding a single variable both above and below, such as $0 \leq w_i \leq 0.10$ (long-only, capped position size).

- Equality constraints — full investment ($\sum w_i = 1$), dollar-neutrality for a long/short book ($\sum w_i = 0$), or a fixed target for a specific factor exposure.

- Inequality constraints — maximum leverage ($\sum |w_i| \leq L$), a minimum expected return, or a maximum tracking-error budget relative to a benchmark.
- Box constraints — long-only ($w_i \geq 0$), position-size caps ($w_i \leq w_{\text{max}}$), or sector-level exposure bands.

Common mistake — Treating every constraint as economically free.

Every constraint added to a problem can only leave the optimal objective value the same or make it worse relative to the unconstrained optimum — it can never improve it, because the optimiser is being told to ignore part of the search space it might otherwise have used. Constraints are frequently necessary (regulatory limits, mandate rules, liquidity realities) and often improve out-of-sample performance despite this, precisely because they compensate for estimation error elsewhere — but they are never economically free, and a long, unexamined list of constraints can silently turn an optimiser into little more than a complicated way of re-deriving the constraints themselves.

1.3 The Feasible Region and the Optimal Solution

The feasible region is the set of every decision-variable value that satisfies all constraints simultaneously — the shaded region in Figure 1.1. The optimiser never considers points outside it, no matter how attractive their objective value would be. The optimal solution is the point (or, in degenerate cases, the set of points) within the feasible region that gives the objective function its best value. If the feasible region is empty — the constraints contradict one another, such as requiring both $w_i \geq 0.30$ and $w_i \leq 0.20$ for the same asset — the problem is infeasible and no solution exists at all, regardless of how the objective is written.

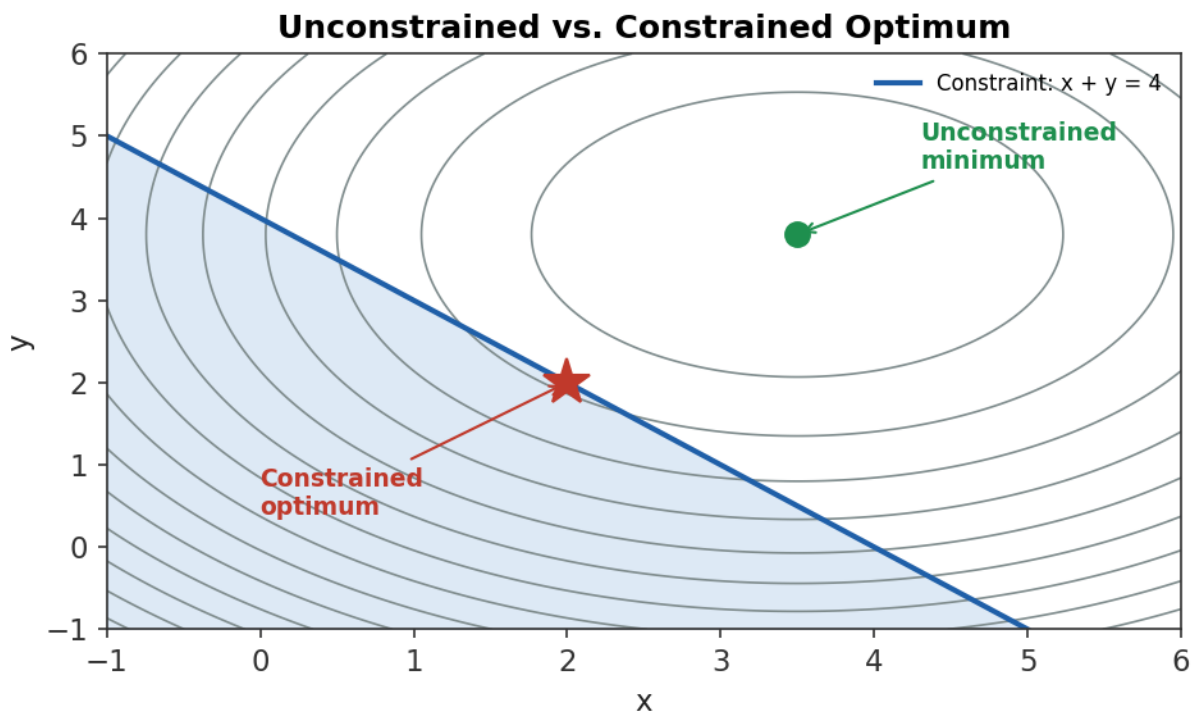


Figure 1.2 — The unconstrained minimum of a bowl-shaped objective (green) sits away from the feasible region defined by a linear constraint; the constrained optimum (red star) is instead the best point available within that region — never better than the unconstrained optimum, and here strictly worse.

Tip — Always check feasibility before trusting a solver's output.

A solver that reports “infeasible” is telling you something true and useful about your constraint set, not failing. A far more dangerous failure mode is a solver that silently returns a numerically “optimal” answer for a badly scaled or nearly-infeasible problem — always sanity-check that a returned solution actually satisfies every constraint to a reasonable tolerance before using it.

Part 2 — Unconstrained and Constrained Optimisation

With the vocabulary of Part 1 in place, this part covers how an optimum is actually found: first in the simple case where no constraints apply, then in the general case where they do, via the single most important idea in constrained optimisation — the Lagrange multiplier.

2.1 Unconstrained Optimisation and Optimality Conditions

In an unconstrained problem, the optimum of a smooth function occurs where its gradient — the vector of partial derivatives — is exactly zero. This is the first-order condition: at a maximum or minimum, there is no direction in which moving slightly would improve the objective, so every partial derivative must vanish simultaneously.

$$\text{First-order condition: } \nabla f(w^*) = 0$$

The first-order condition alone cannot distinguish a minimum from a maximum or from a saddle point — all three have zero gradient. The second-order condition resolves this using the Hessian matrix (the matrix of second partial derivatives): if the Hessian is positive definite at the stationary point, it is a local minimum; if negative definite, a local maximum; if indefinite, a saddle point. This is why, in practice, most optimisation routines do not solve $\nabla f(w) = 0$ analytically for anything beyond the simplest problems — they instead search iteratively, using the gradient to decide which direction to move in next.

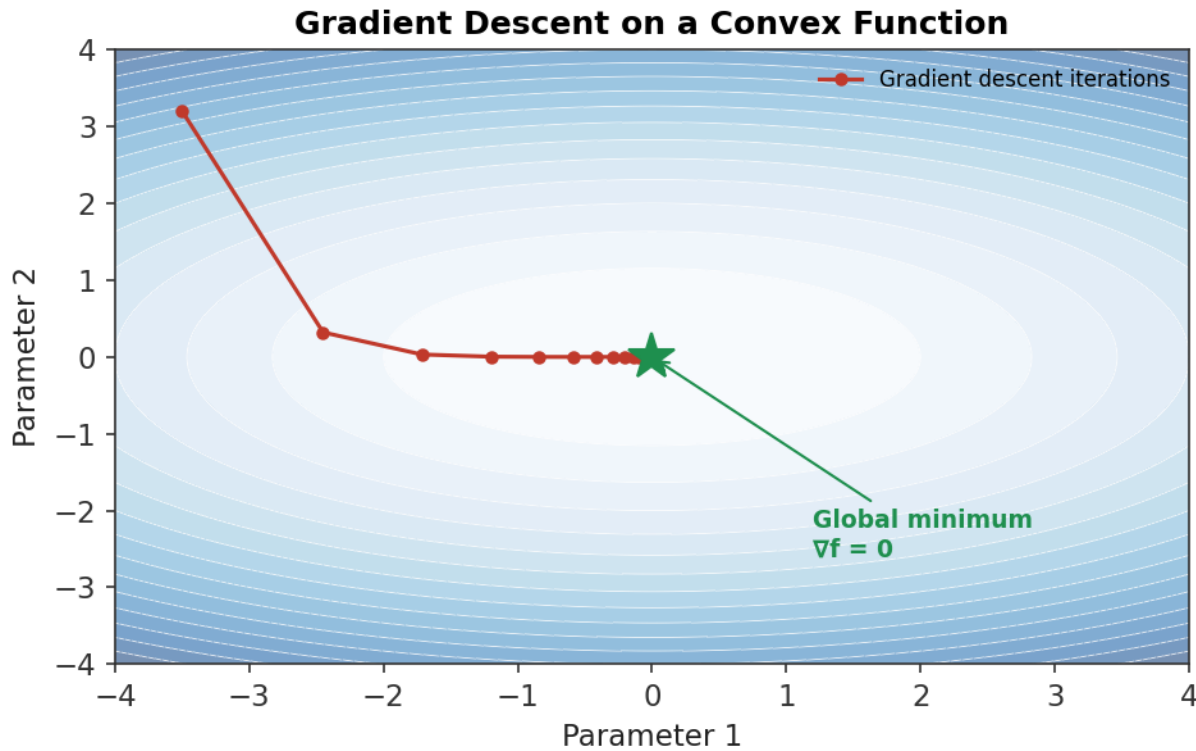


Figure 2.1 — Gradient descent iteratively follows the negative gradient downhill across the contours of a convex objective, converging to the point where the gradient is zero — the global minimum.

```
# Minimal gradient-descent sketch for an unconstrained quadratic objective
import numpy as np

def f(w):      return w @ A @ w - b @ w      # objective, A positive-definite
def grad(w):   return 2 * A @ w - b          # analytic gradient

w = np.zeros(n)                # start anywhere
lr = 0.05
for _ in range(200):
    w = w - lr * grad(w)        # step opposite the gradient
# w now approximates the unconstrained optimum w* where grad(w*) = 0
```

Common mistake — Assuming a solver that converges has found the global optimum.

Convergence means the algorithm stopped moving — it says nothing, by itself, about whether it stopped at a global optimum, a merely local one, or a saddle point, unless the underlying problem is known to be convex (Part 3). Gradient-based methods on a non-convex objective can converge cleanly and confidently to a poor local optimum, and will not report this as a problem.

2.2 Constrained Optimisation and Lagrange Multipliers

The Lagrange multiplier method converts a constrained problem into an unconstrained one by folding each constraint into the objective, weighted by a new variable — the multiplier — that the optimiser also solves for. For a problem of maximising $f(w)$ subject to a single equality constraint $g(w) = 0$, the Lagrangian is:

$$L(w, \lambda) = f(w) - \lambda \cdot g(w)$$

Setting the gradient of L with respect to both w and λ to zero recovers, simultaneously, the original constraint (from $\partial L / \partial \lambda = 0$, which just restates $g(w) = 0$) and a geometric condition on w : at the optimum, the gradient of the objective must be parallel to the gradient of the constraint, $\nabla f(w) = \lambda \nabla g(w)$.

Geometrically, this is the point where the highest-value contour of the objective just touches the constraint curve — any further along the constraint and the objective would be lower, so the contour and the constraint are tangent, not crossing, exactly at the optimum.

Lagrange Multipliers: Tangency at the Constrained Optimum

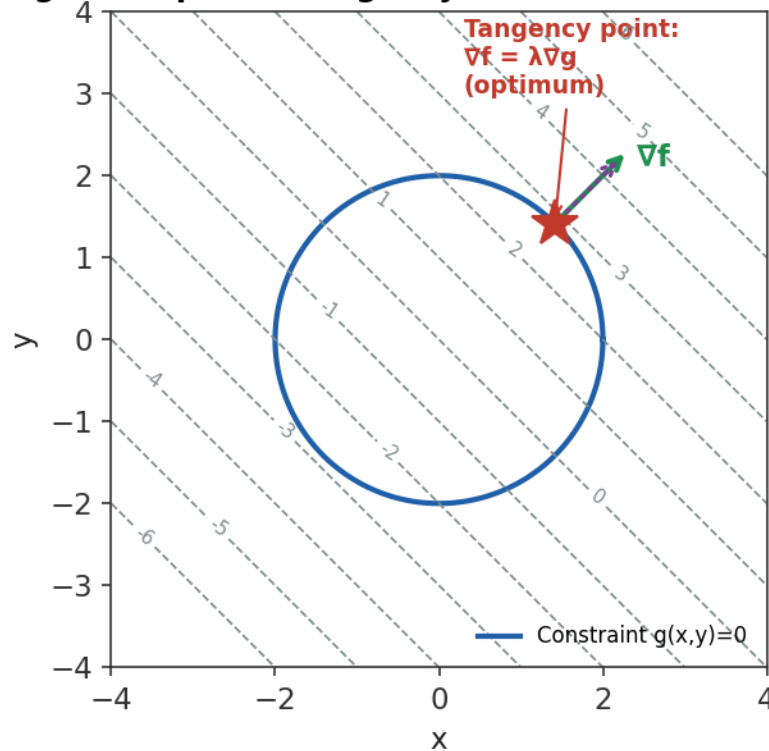


Figure 2.2 — At the constrained optimum, the objective's contour line is tangent to the constraint curve — their gradients point in the same (or exactly opposite) direction, which is precisely the condition $\nabla f = \lambda \nabla g$.

Worked example — Solving a Lagrangian by hand.

Maximise $f(x, y) = x + y$ subject to the constraint $g(x, y) = x^2 + y^2 - 4 = 0$ (i.e., (x, y) lies on a circle of radius 2). The Lagrangian is $L = x + y - \lambda(x^2 + y^2 - 4)$. Setting $\partial L / \partial x = 1 - 2\lambda x = 0$ and $\partial L / \partial y = 1 - 2\lambda y = 0$ gives $x = y = 1/(2\lambda)$. Substituting into the constraint: $2 \times (1/(2\lambda))^2 = 4$, which solves to $\lambda = 1/(2\sqrt{2})$. This gives $x = y = \sqrt{2}$, so the constrained maximum is $f = \sqrt{2} + \sqrt{2} = 2\sqrt{2} \approx 2.83$ — matching the tangency point shown in Figure 2.2.

The multiplier λ itself carries an economically important interpretation beyond just being an algebraic device: it measures the shadow price of the constraint — how much the optimal objective value would improve if the constraint were relaxed by one unit. In a portfolio context, the multiplier on a leverage constraint tells you, at the margin, exactly how much expected return (or utility) is being left on the table by that specific limit — valuable information when deciding whether to request a mandate change.

2.3 The Karush-Kuhn-Tucker (KKT) Conditions (awareness)

Awareness-level: understand what the KKT conditions are for and why they generalise Lagrange multipliers, not how to derive or apply them by hand to a novel problem, which belongs to Depth 3+.

Real portfolio problems rarely have only equality constraints — long-only, position-cap, and leverage limits are all inequalities. The Karush-Kuhn-Tucker (KKT) conditions extend the Lagrangian idea to handle inequality constraints as well, introducing a non-negative multiplier for each inequality and an additional “complementary slackness” condition: at the optimum, either an inequality constraint is exactly binding (holds with equality) or its multiplier is zero — never both non-binding and non-zero at once. Every modern quadratic-programming and convex-optimisation solver is, under the hood, searching for a point that satisfies the KKT conditions for the problem it has been given; when a solver reports success, it is reporting that it found a KKT point.

Prerequisite gate — KKT conditions are foundational but not something you need to derive by hand at this level.

Understanding that a binding constraint has a nonzero multiplier (a real economic cost) while a non-binding constraint has a zero multiplier (no cost, because the optimiser wasn't going to violate it anyway) is enough to interpret solver output sensibly at this level. Deriving and implementing a KKT-based solver from scratch is a Depth 3+ / numerical-methods topic.

Part 3 — Convex Optimisation and Quadratic Programming

Sections 2.1 and 2.3 both hinted at the same caveat: gradient conditions and KKT points identify candidates for an optimum, but not necessarily the best one, unless the underlying problem has a special structural property. This part covers that property — convexity — and the single most important convex problem class in portfolio construction: quadratic programming.

3.1 Convex Sets and Convex Functions

A set is convex if the straight line segment connecting any two points inside it lies entirely inside it as well — no dents, holes, or disconnected pieces. A function is convex if the straight line (chord) connecting any two points on its graph lies on or above the graph everywhere between them; intuitively, it curves upward everywhere, like a bowl, never wiggling down into a separate local dip.

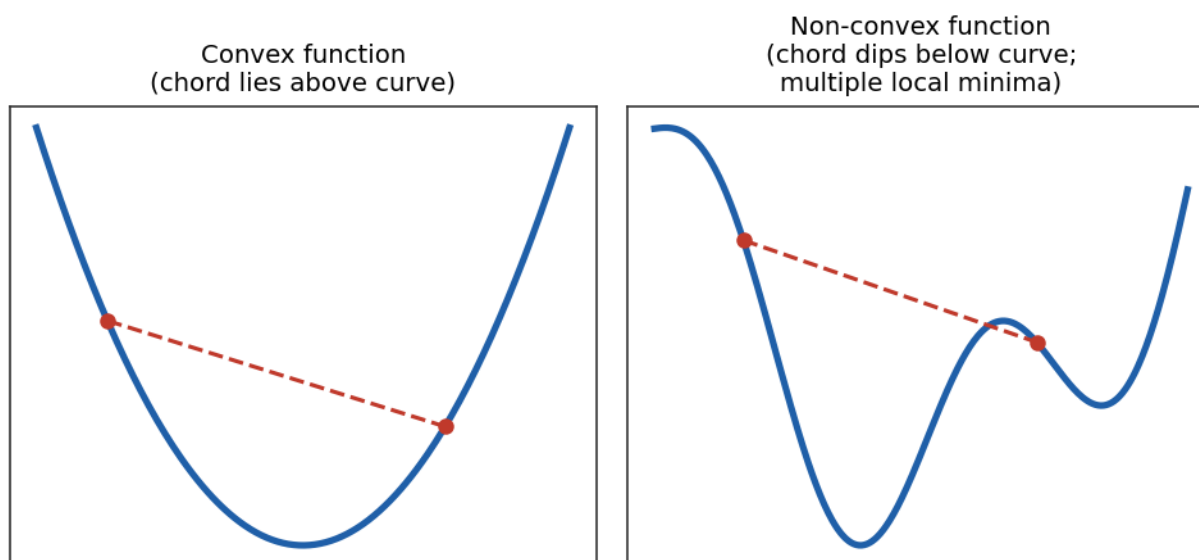


Figure 3.1 — A convex function's chord always sits above the curve (left); a non-convex function's chord can dip below it, and the function itself may have several separate local minima (right).

Formal definition: f is convex if $f(\vartheta x + (1-\vartheta)y) \leq \vartheta f(x) + (1-\vartheta)f(y)$ for all $\vartheta \in [0,1]$

3.2 Why Convexity Is the Property That Makes Optimisation Tractable

Convexity matters because it delivers a guarantee that almost nothing else in optimisation does: for a convex objective over a convex feasible region, any local optimum is automatically the global optimum. There is no possibility of a solver getting trapped in a plausible-looking but suboptimal corner of the search space — if it finds a point satisfying the first-order (or KKT) conditions, that point is provably the best point in the entire feasible region, not just the best point nearby.

Portfolio variance, $w^T \Sigma w$, is a convex (in fact quadratic) function of w whenever Σ is positive semi-definite — which every valid covariance matrix is by construction. This single fact is why mean-variance optimisation is well-posed and solvable in general, while many more exotic objectives (certain non-linear risk measures, some machine-learning-based scoring functions used directly as optimisation targets) are not convex and require the more specialised, less-guaranteed methods flagged in Advanced Extensions.

Common mistake — Assuming convex optimisation automatically produces an economically sensible solution.

Convexity is a guarantee about the mathematics of the search, not about the economics of the answer. A convex optimisation problem built on garbage inputs — badly estimated expected returns, a covariance matrix distorted by a short lookback window — will still converge cleanly and confidently to the global optimum of that flawed problem. Convexity guarantees you found the right answer to the question you asked; it says nothing about whether you asked a sensible question.

3.3 Quadratic Programming

A quadratic program (QP) is an optimisation problem with a quadratic objective and linear constraints — exactly the structure of standard mean-variance portfolio optimisation, where the objective (variance, or the mean-variance utility of Section 1.1) is quadratic in the weights and constraints such as full investment, long-only, and position limits are all linear.

$$\text{Standard QP form: minimise } \frac{1}{2} w^T \Sigma w - q^T w \text{ subject to } Aw \leq b, Cw = d$$

Here Σ is the (quadratic) covariance term, q typically encodes expected returns, and the linear constraint matrices A and C encode every inequality and equality restriction — leverage caps, box constraints, full-investment, and so on — in a single unified representation. Because the objective is convex whenever Σ is positive semi-definite and the constraints are linear (and therefore define a convex feasible region, a polytope), a QP inherits the global-optimality guarantee of Section 3.2, and dedicated QP solvers can find that global optimum extremely efficiently even for portfolios with hundreds or thousands of assets.

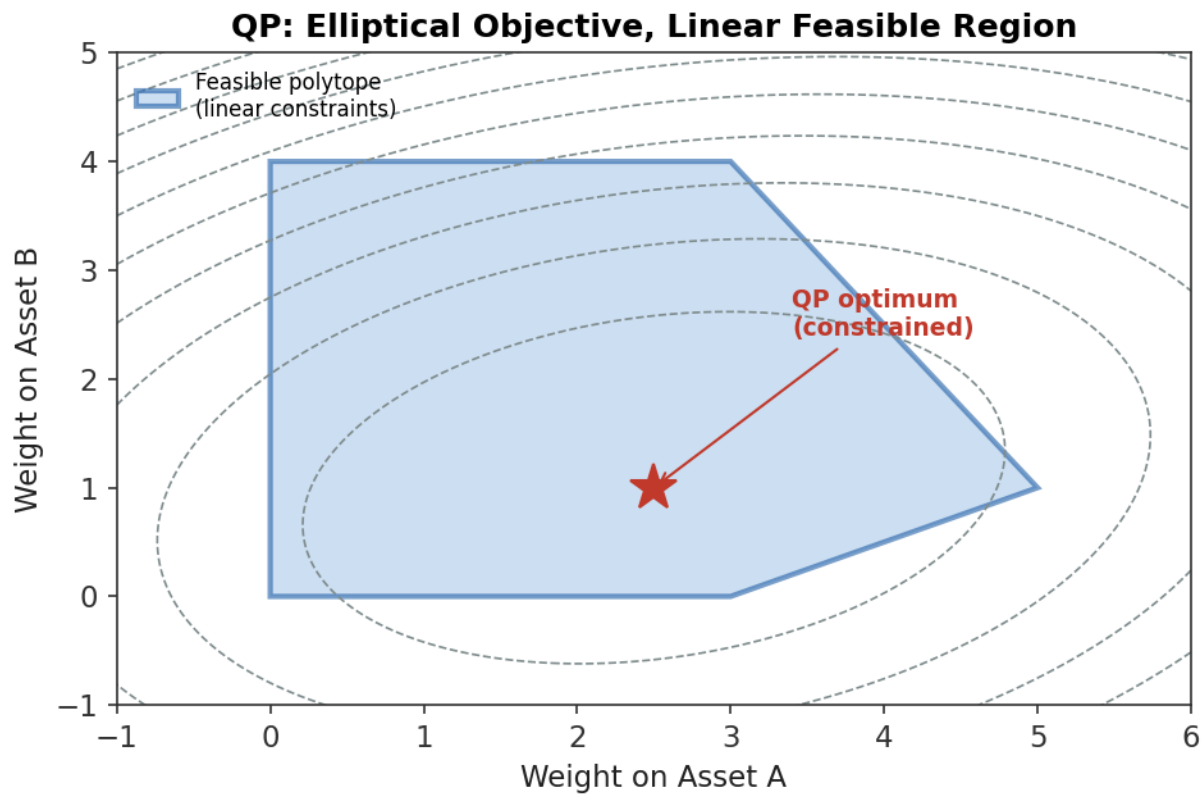


Figure 3.3 — A quadratic (elliptical-contour) objective over a linear feasible region (polytope). The optimum sits either at an interior tangency point, as shown, or at a vertex of the polytope if the unconstrained optimum falls outside the feasible region.

```
# Minimal QP-style mean-variance solve using a generic convex solver
import numpy as np, cvxpy as cp

n = len(mu)
w = cp.Variable(n)
risk_aversion = 4.0
objective = cp.Maximize(mu @ w - 0.5 * risk_aversion * cp.quad_form(w, Sigma))
constraints = [cp.sum(w) == 1, w >= 0, w <= 0.10]  # full investment, long-only, 10%
cap
prob = cp.Problem(objective, constraints)
prob.solve()
optimal_weights = w.value
```

Tip — Recognise when a problem is (or can be reformulated as) a QP.

Mean-variance optimisation, minimum-variance portfolios, tracking-error minimisation against a benchmark, and many turnover-penalised rebalancing problems are all QPs, or become QPs with a small reformulation — recognising this lets you reach directly for a fast, reliable, off-the-shelf QP solver rather than a slower general-purpose non-linear optimiser.

Part 4 — Mean-Variance Optimisation and the Efficient Frontier

This part applies Parts 1–3 to the problem that motivates most of this module: Harry Markowitz's mean-variance framework, the foundation on which almost all quantitative portfolio construction is still built more than seventy years after its introduction.

4.1 The Markowitz Mean-Variance Framework

The mean-variance framework treats a portfolio as fully characterised, for optimisation purposes, by two numbers: its expected return and its variance (or equivalently, its volatility, the square root of variance). Given a vector of expected returns μ and a covariance matrix Σ , portfolio expected return and variance as a function of weights w are:

$$\text{Portfolio return: } \mu_p = w^T \mu \quad \text{Portfolio variance: } \sigma_p^2 = w^T \Sigma w$$

The framework's central insight is that these two numbers, not the full distribution of returns, are (under reasonable assumptions) sufficient to rank portfolios by desirability: a rational, risk-averse investor prefers more expected return for the same risk, and less risk for the same expected return. This reduces the entire portfolio-construction problem to a well-posed optimisation — exactly the QP of Section 3.3 — rather than an intractable search over the space of every possible return distribution.

Worked example — Three-asset portfolio variance.

Three assets have volatilities $\sigma = (18\%, 12\%, 9\%)$ and a correlation matrix with $\rho_{12} = 0.4$, $\rho_{13} = 0.1$, $\rho_{23} = 0.2$. For equal weights $w = (1/3, 1/3, 1/3)$: the covariance terms are $\Sigma_{12} = 0.18 \times 0.12 \times 0.4 = 0.00864$, $\Sigma_{13} = 0.18 \times 0.09 \times 0.1 = 0.00162$, $\Sigma_{23} = 0.12 \times 0.09 \times 0.2 = 0.00216$. Portfolio variance = $\sum_i w_i^2 \sigma_i^2 + 2 \sum_{i < j} w_i w_j \Sigma_{ij} = (1/9)(0.0324 + 0.0144 + 0.0081) + 2(1/9)(0.00864 + 0.00162 + 0.00216) = 0.00610 + 0.00272 = 0.00882$, so portfolio volatility $\approx 9.4\%$ — notably lower than any single asset's volatility, purely from diversification.

4.2 The Efficient Frontier

Solving the mean-variance QP repeatedly — once for every target level of expected return, each time minimising variance subject to that return target and the full-investment constraint — traces out a curve in risk-return space called the efficient frontier: the set of portfolios offering the lowest possible variance for each level of expected return (equivalently, the highest possible return for each level of variance). Every portfolio not on this curve is dominated — there exists some portfolio on the frontier with either the same return and lower risk, or the same risk and higher return.

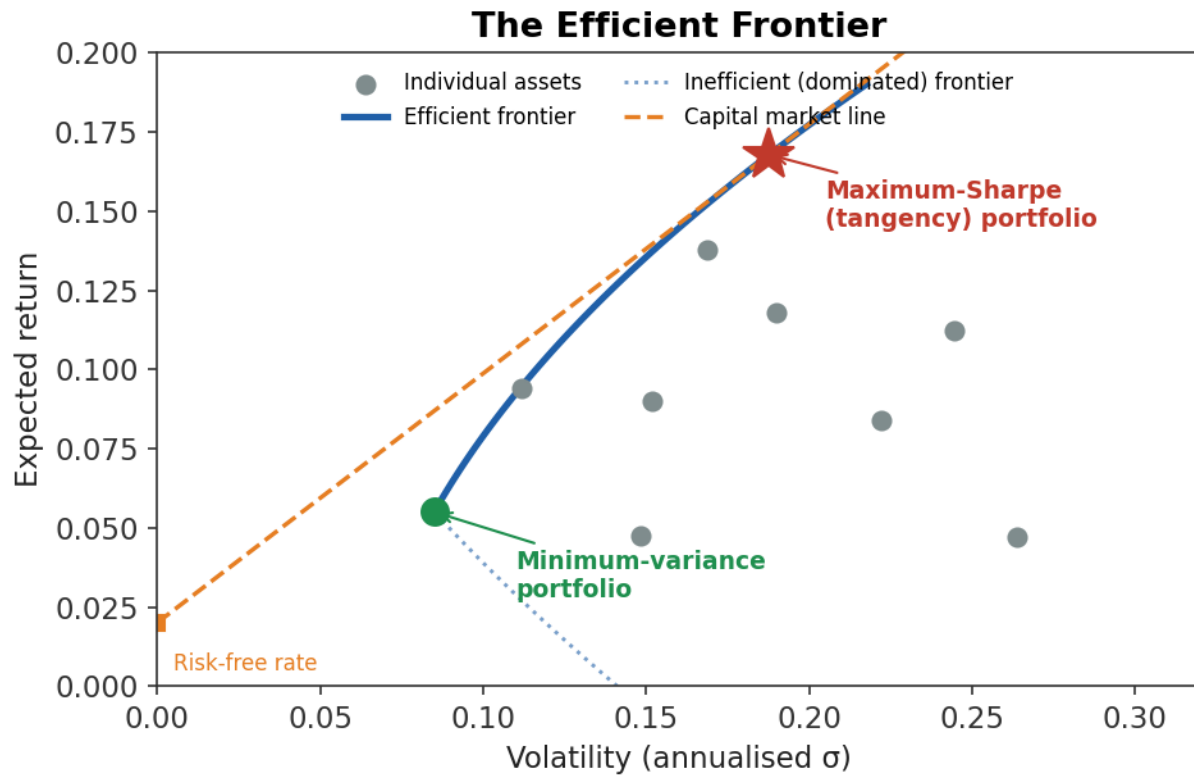


Figure 4.1 — The efficient frontier (solid blue) traced from the minimum-variance portfolio upward. Individual assets (grey dots) sit inside the frontier because they are undiversified; the capital market line (orange) is tangent to the frontier at the maximum-Sharpe portfolio.

Correlation drives the shape and the benefit of the frontier directly. When two assets are perfectly correlated ($\rho = 1$), there is no diversification benefit at all and the “frontier” between them is just a straight line — a weighted average of their individual risk and return with nothing extra. As correlation falls, and especially once it turns negative, the frontier bows further to the left: it becomes possible to combine the two assets into a portfolio with lower risk than either asset held alone.

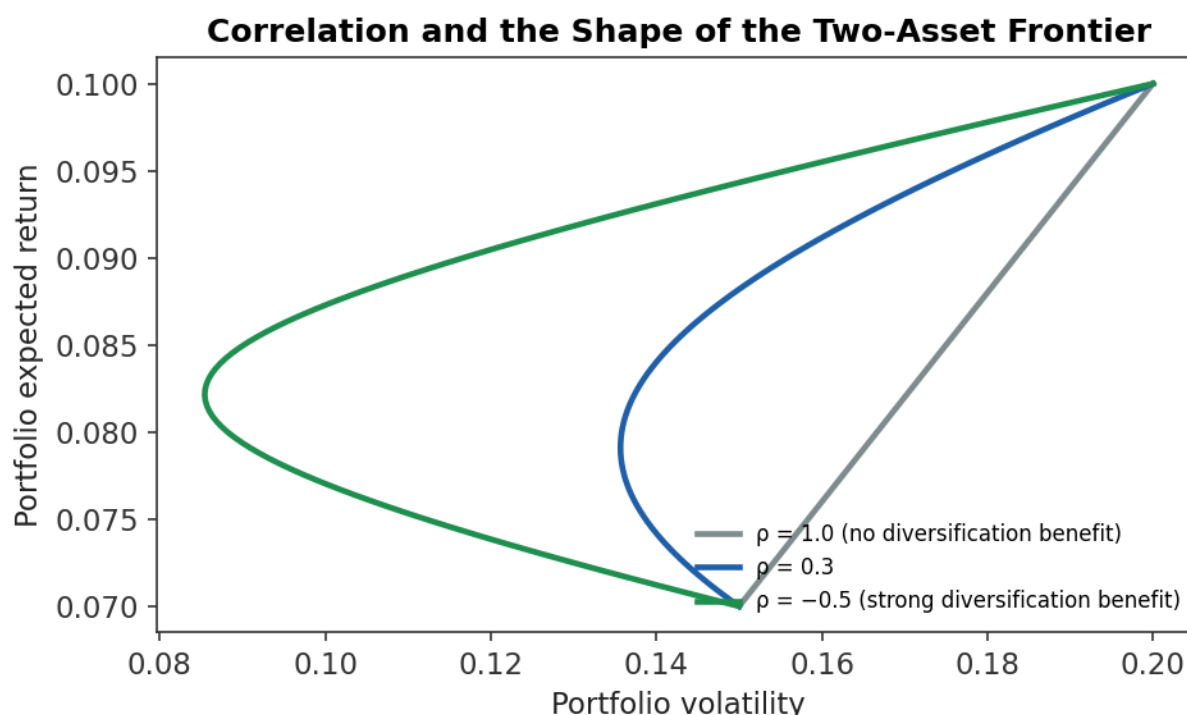


Figure 4.2 — The two-asset frontier bows further left as correlation falls: at $\rho = 1$ there is no diversification benefit (straight line); at $\rho = -0.5$ the combined portfolio can achieve materially lower risk than either asset individually.

Common mistake — Optimising directly for historical Sharpe ratio and overfitting to the backtest period.

Estimating μ and Σ from the same historical window used to evaluate the resulting portfolio, then reporting that portfolio's in-sample Sharpe ratio as if it were a forward-looking expectation, is a direct form of overfitting: the optimiser has been explicitly rewarded for exploiting whatever noise happened to occur in that specific sample. Out-of-sample performance of a naively mean-variance-optimised portfolio is routinely far worse than its in-sample Sharpe ratio would suggest — a central motivation for the regularisation and robust-optimisation techniques in Part 6.

4.3 Minimum-Variance and Maximum-Sharpe Portfolios

Two single points on the efficient frontier deserve special names because they answer two very common practical questions. The minimum-variance portfolio is the leftmost point on the frontier — the portfolio with the lowest possible risk of any fully invested combination of the available assets, found by minimising $w^T \Sigma w$ subject only to $\Sigma w_i = 1$ (notably, this portfolio does not depend on μ at all, which makes it far less sensitive to the expected-return estimation error flagged throughout this module).

Minimum-variance portfolio: minimise $w^T \Sigma w$ subject to $\Sigma w_i = 1$

The maximum-Sharpe (tangency) portfolio is the point where a straight line drawn from the risk-free rate is tangent to the frontier — the single portfolio of risky assets that, combined with borrowing or lending at the

risk-free rate, produces the best possible risk-return trade-off along the entire capital market line shown in Figure 4.1.

Maximum-Sharpe portfolio: maximise $(w^T\mu - r_f) / \sqrt{w^T\Sigma w}$ subject to $\Sigma w_i = 1$

Common mistake — Using sample covariance matrices without considering estimation error.

A sample covariance matrix estimated from a short history is itself a noisy estimate of the true, unobservable covariance matrix, and the maximum-Sharpe portfolio in particular is highly sensitive to this noise because it also depends on μ , the single hardest quantity in finance to estimate reliably. Small, statistically insignificant differences in estimated expected returns can flip the maximum-Sharpe optimiser between wildly different “optimal” portfolios — a well-known instability that motivates both the minimum-variance portfolio's popularity in practice (it sidesteps μ entirely) and the shrinkage techniques of Section 6.2.

Part 5 — Risk-Based Portfolio Construction

Mean-variance optimisation's sensitivity to expected-return estimates has motivated an entire family of alternative approaches that sidestep μ altogether and allocate purely on the basis of risk. This part covers the two most widely used: risk parity and its more general form, Equal Risk Contribution.

5.1 Risk Parity

Risk parity allocates capital so that each asset contributes an equal amount of risk to the portfolio, rather than an equal amount of capital (as in a naive $1/N$ equal-weight portfolio) or the concentrated, return-forecast-driven weights a mean-variance optimiser can produce. In its simplest form, an asset's weight is set inversely proportional to its own volatility:

Simple (volatility-only) risk parity: $w_i \propto 1 / \sigma_i$

The logic is direct: a low-volatility asset (say, short-dated government bonds) needs a large capital allocation to contribute a meaningful amount of risk to the portfolio, while a high-volatility asset (say, equities or commodities) needs only a small allocation to contribute the same amount. A naive equal-capital allocation across an asset class with very different volatilities — the classic 60/40 stock/bond portfolio being the textbook example — ends up with the vast majority of its risk coming from the single highest-volatility asset, despite looking diversified by capital.

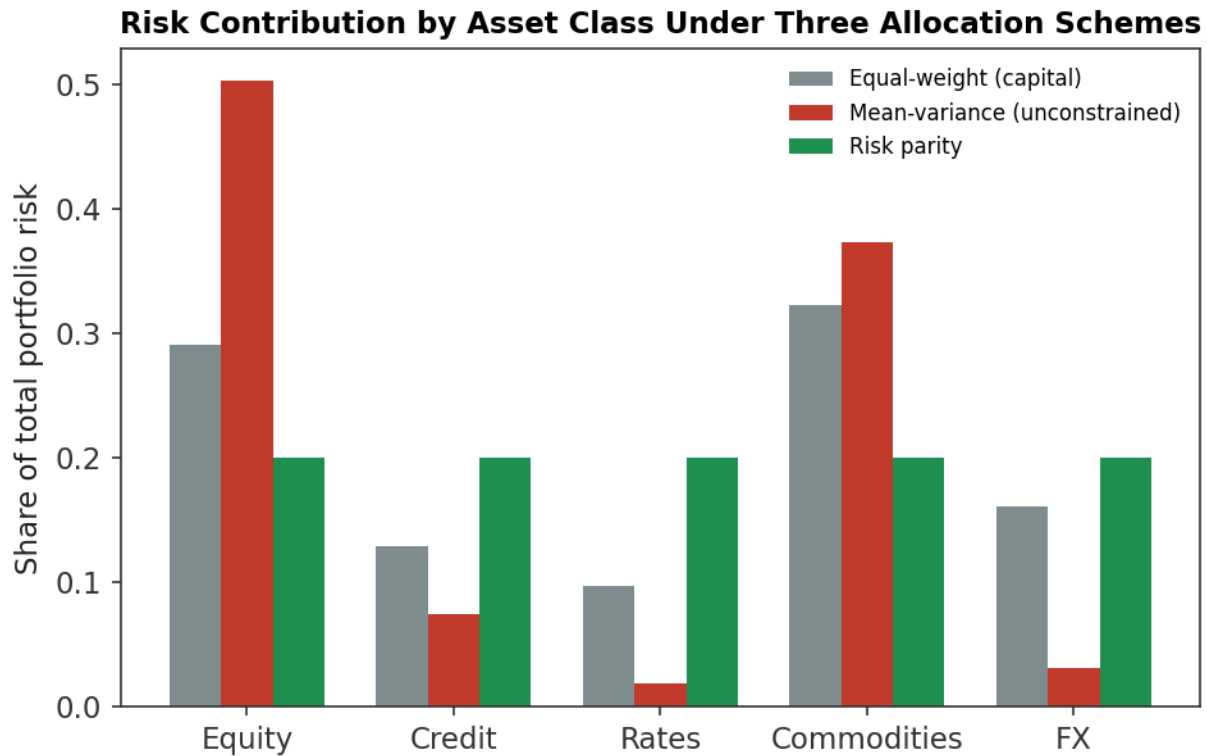


Figure 5.1 — Under an equal-weight (by capital) allocation, equities and commodities dominate total portfolio risk despite each receiving only 20% of capital; under risk parity, every asset class contributes an equal 20% risk share instead.

Worked example — Simple risk-parity weights.

Two assets have volatilities $\sigma_1 = 20\%$ and $\sigma_2 = 5\%$. Simple risk parity sets $w_i \propto 1/\sigma_i$, so unnormalised weights are $1/0.20 = 5$ and $1/0.05 = 20$; normalising to sum to 1 gives $w_1 = 5/25 = 20\%$ and $w_2 = 20/25 = 80\%$. The low-volatility asset receives four times the capital allocation of the high-volatility asset — the opposite of what an equal-weight (50/50) allocation would do, and precisely calibrated so that, ignoring correlation, both assets contribute equally to total portfolio risk.

5.2 Equal Risk Contribution (ERC)

Simple risk parity ignores correlations between assets, which is a reasonable approximation for a small number of largely uncorrelated asset classes but breaks down as a portfolio grows and its assets become more interrelated. Equal Risk Contribution generalises the idea properly, defining each asset's marginal risk contribution using the full covariance matrix and then solving explicitly for the weights that equalise every asset's contribution to total portfolio risk.

$$\text{Marginal contribution to risk: } MRC_i = (\Sigma w)_i / \sigma_p \quad \text{Risk contribution: } RC_i = w_i \cdot MRC_i$$

The ERC objective is to find weights such that $RC_i = \sigma_p/n$ for every asset i — an equal $1/n$ share of total portfolio risk each. Unlike simple risk parity, this is not a closed-form calculation for more than two assets; it requires an iterative numerical solve, since each asset's risk contribution depends on every other asset's weight through the off-diagonal covariance terms. ERC portfolios sit in an interesting middle ground: like the

minimum-variance portfolio, they require no expected-return input at all, but unlike simple $1/\sigma$ risk parity, they correctly account for the risk-reducing (or risk-amplifying) effect of correlation between every pair of assets.

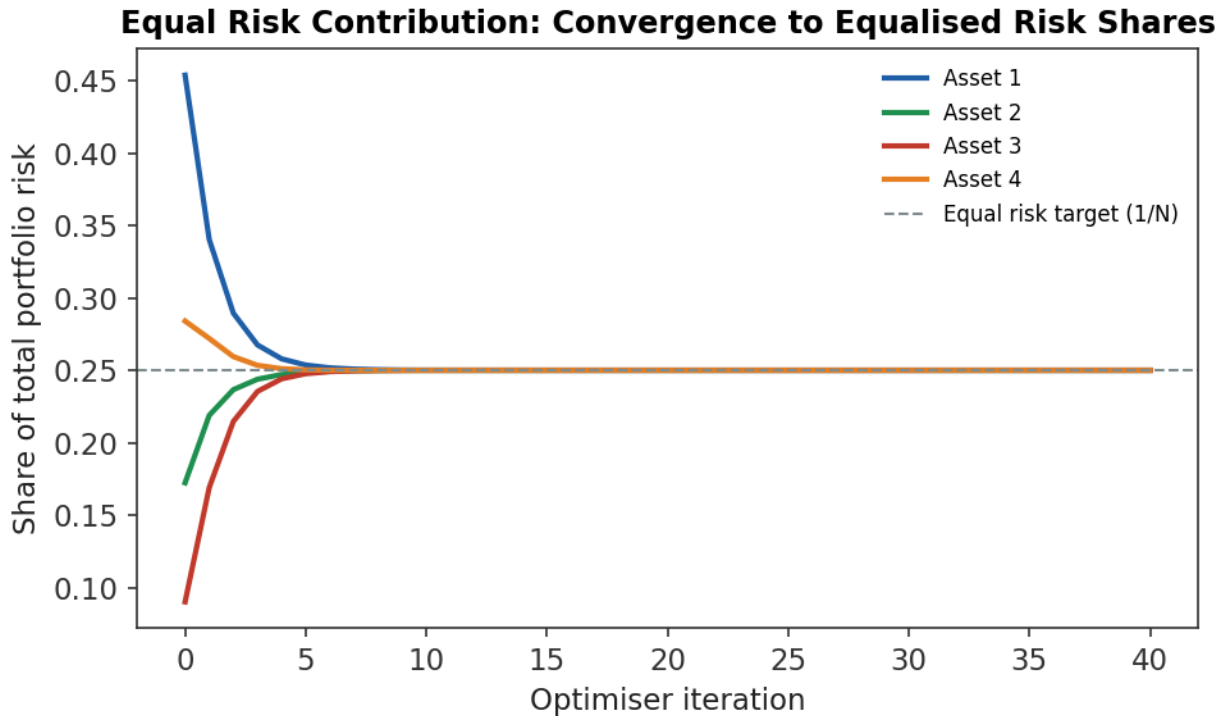


Figure 5.2 — An ERC optimiser iteratively adjusts weights until every asset's share of total portfolio risk converges to the equal $1/n$ target (dashed line) — here, four assets each converging to a 25% risk share.

```
# Minimal iterative ERC solve (risk-budgeting update)
import numpy as np

w = np.ones(n) / n          # start equal-weight
for _ in range(200):
    port_vol = np.sqrt(w @ Sigma @ w)
    mrc = (Sigma @ w) / port_vol  # marginal risk contribution per asset
    rc = w * mrc                 # risk contribution per asset
    target = port_vol / n        # equal risk-contribution target
    w = w * (target / rc) ** 0.5  # multiplicative update toward target
    w = w / w.sum()             # renormalise to full investment
# w now approximately equalises each asset's contribution to total risk
```

Tip — ERC is a natural default when expected-return forecasts are weak or absent.

Because neither minimum-variance nor ERC portfolios use μ at all, both are far more stable across re-estimation periods than a full mean-variance optimum — a useful default whenever the available return forecasts are low-confidence, short-horizon, or simply unavailable, such as for a long-term strategic asset-allocation decision across broad asset classes.

Part 6 — Making Optimisation Realistic: Costs, Stability, and Robustness

Every optimisation covered so far has been, implicitly, a single-period, frictionless, perfect-information exercise. This part closes that gap, covering the three adjustments that separate a textbook optimiser from one that survives contact with a real portfolio: the cost of trading into the new solution, the instability of the inputs feeding it, and the risk that those inputs are simply wrong.

6.1 Transaction Costs and Turnover Constraints

Re-optimising a portfolio from scratch every period, with no reference to the current holdings, treats trading as free — an assumption that is never true and, as covered in the Market Microstructure module, least true for exactly the large, urgent trades that a naive re-optimisation tends to generate. Two standard remedies exist. The first folds an explicit transaction-cost penalty directly into the objective function, so the optimiser only trades when the expected benefit exceeds the cost of getting there:

$$\text{Cost-aware objective: maximise } w^T \mu - (\lambda/2) \cdot w^T \Sigma w - \kappa \cdot \|w - w_{\text{prev}}\|$$

where w_{prev} is the current portfolio, $\|w - w_{\text{prev}}\|$ measures total trading required, and κ scales the assumed per-unit cost of that trading (spread, market impact, commission). The second remedy imposes a hard turnover constraint — a maximum allowed change in any position, or in total portfolio turnover, per rebalancing period — which is simpler to implement and reason about, at the cost of being a blunter instrument than a cost-aware objective that can trade more when the opportunity clearly justifies it.

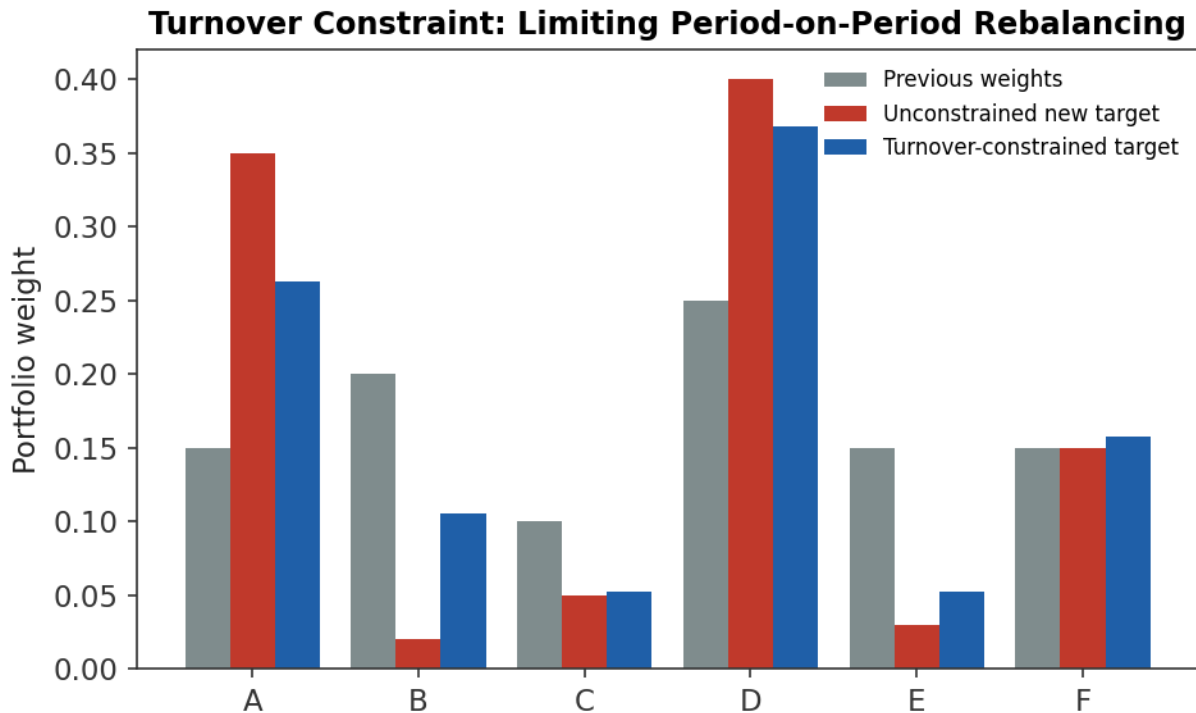


Figure 6.1 — An unconstrained re-optimisation (red) would require large trades away from current holdings (grey); a turnover constraint (blue) caps how far any single position can move per period, trading off responsiveness against transaction-cost drag.

Common mistake — Ignoring transaction costs, turnover, liquidity, and market-impact constraints entirely.

A backtest that re-optimises every period with no cost or turnover penalty is implicitly assuming free, instantaneous, unlimited-size trading — exactly the assumption the Market Microstructure module shows to be false. The resulting backtested performance can look excellent purely because the optimiser is being rewarded for chasing small, noisy changes in estimated μ and Σ period to period, generating turnover that would be prohibitively expensive (and often simply undeliverable, given real market depth) outside the backtest.

6.2 Regularisation and Shrinkage

The instability flagged repeatedly throughout this module — sample covariance and, especially, sample expected-return estimates producing extreme, unstable “optimal” weights — has a standard statistical remedy: regularisation, more commonly called shrinkage in this context. The idea is to deliberately bias a noisy sample estimate toward a simpler, lower-variance target, accepting a small, known amount of bias in exchange for a large reduction in estimation-error-driven instability.

$$\text{Shrinkage estimator: } \hat{\Sigma}_{shrunk} = \alpha \cdot \hat{\Sigma}_{sample} + (1-\alpha) \cdot \hat{\Sigma}_{target}, \quad 0 \leq \alpha \leq 1$$

Common shrinkage targets for a covariance matrix include a constant-correlation matrix, a single-factor (market) model, or simply a scaled identity matrix; the Ledoit-Wolf estimator is the best-known method for choosing the shrinkage intensity α in a statistically principled way rather than guessing it. The same logic applies to expected-return estimates, most famously in the Black-Litterman model (Advanced Extensions), which shrinks noisy historical or subjective return estimates toward the returns implied by current market-capitalisation weights.

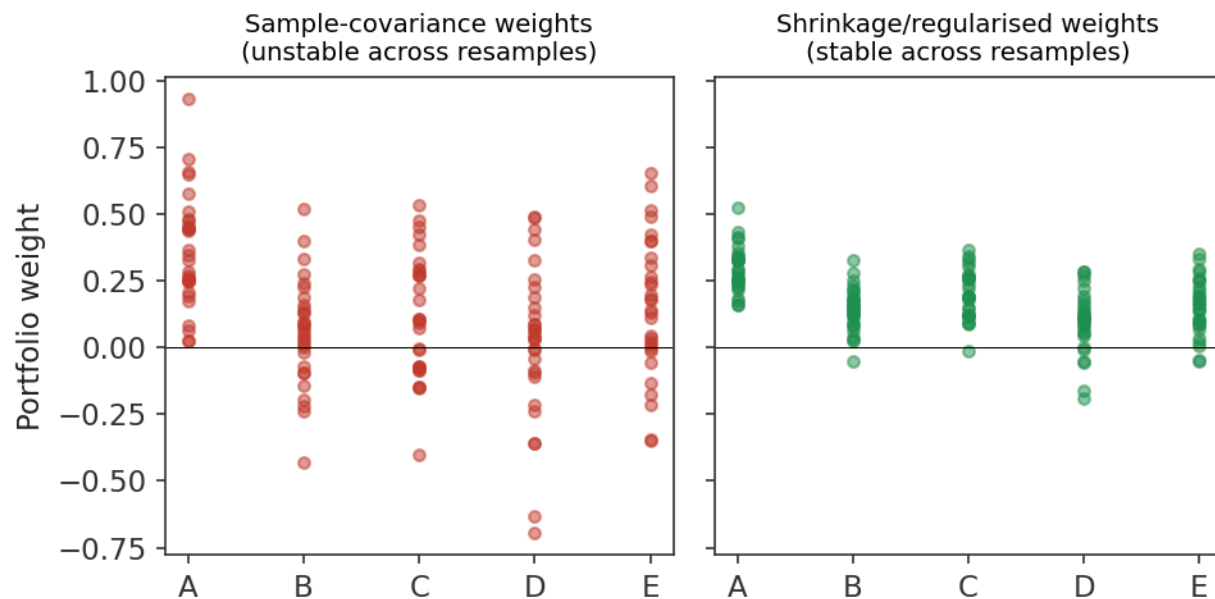


Figure 6.2 — Portfolio weights re-estimated across thirty resampled historical windows: raw sample-covariance-based weights (left) swing wildly asset to asset, while shrinkage-regularised weights (right) stay far more stable, even though both use the same underlying data.

Worked example — A simple shrinkage blend.

A sample covariance matrix estimate $\hat{\Sigma}_{\text{sample}}$ is blended with a simpler target $\hat{\Sigma}_{\text{target}}$ (say, a constant-correlation matrix) using $\alpha = 0.4$: $\hat{\Sigma}_{\text{shrunk}} = 0.4 \times \hat{\Sigma}_{\text{sample}} + 0.6 \times \hat{\Sigma}_{\text{target}}$. If a particular off-diagonal covariance term is 0.018 in the noisy sample estimate but only 0.008 under the simpler target structure, the shrunk estimate is $0.4 \times 0.018 + 0.6 \times 0.008 = 0.0072 + 0.0048 = 0.0120$ — pulled substantially toward the more stable target, damping the effect of whatever sample-specific noise produced the original 0.018.

6.3 Robust Optimisation (awareness)

Awareness-level: understand what problem robust optimisation solves and the intuition behind it, not how to formulate or solve a robust or distributionally-robust program from scratch, which belongs to Depth 3+.

Robust optimisation addresses estimation uncertainty from a different angle than shrinkage: rather than producing a single, better point estimate of μ or Σ and optimising against it as if it were exactly correct, robust optimisation defines an uncertainty set — a range of plausible values around the point estimate — and optimises for the worst case within that set. The resulting portfolio may sacrifice some performance under the single most likely scenario, in exchange for far more consistent performance across the range of scenarios that could plausibly be true.

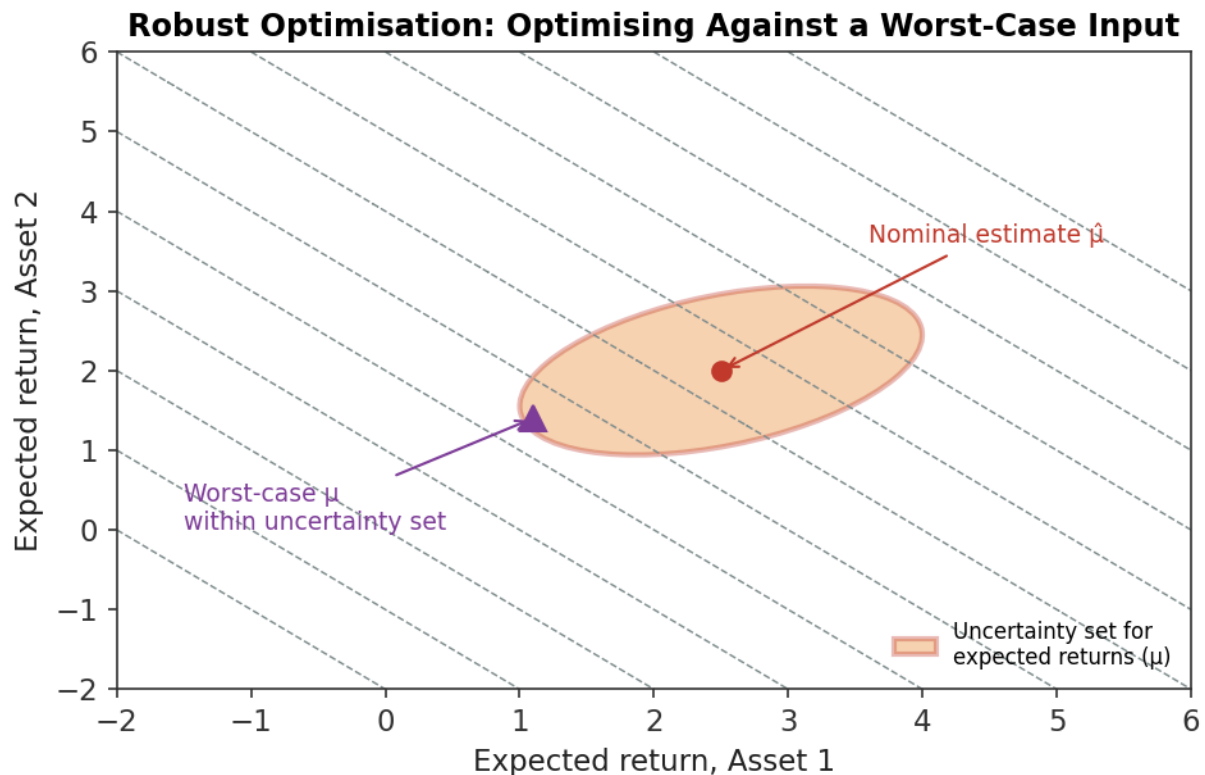


Figure 6.3 — Rather than optimising against a single point estimate $\hat{\mu}$, robust optimisation considers every value inside an uncertainty set around it and optimises for the worst case that could plausibly occur within that set.

Prerequisite gate — Robust optimisation is a Depth 3+ formulation technique.

At this level, the goal is to recognise robust optimisation as the natural response to the same estimation-error problem that motivates shrinkage — approached from the constraint side rather than the input side — and to know it exists as an option. Formulating the uncertainty set correctly and solving the resulting min-max problem efficiently is specialised numerical-optimisation territory covered at Depth 3+.

Practical Exercise

Construct a mean-variance portfolio optimiser for a universe of at least six assets. Calculate the minimum-variance portfolio and the maximum-Sharpe portfolio, plot the efficient frontier, then add constraints for maximum position size, long-only exposure, and portfolio turnover relative to a starting allocation. Compare the unconstrained and constrained solutions.

Suggested approach

- Estimate μ and Σ from historical returns for your chosen universe, and separately compute each asset's standalone volatility and pairwise correlations to sanity-check the covariance matrix by eye before using it.
- Solve the unconstrained QP for a grid of target expected returns to trace the efficient frontier, then identify the minimum-variance point and, given an assumed risk-free rate, the maximum-Sharpe (tangency) point.
- Re-solve with box constraints (e.g., $0 \leq w_i \leq 0.15$) and a turnover constraint relative to an assumed prior $1/N$ allocation, and compare the resulting weights, expected return, and volatility against the unconstrained solution.
- Compute each asset's risk contribution under both the mean-variance and an Equal Risk Contribution solution for the same universe, and compare how concentrated risk is under each.

```
# Skeleton for the practical exercise
import numpy as np, cvxpy as cp

mu, Sigma = estimate_inputs(returns)          # from historical data
n = len(mu)
w = cp.Variable(n)

# 1. Minimum-variance portfolio
prob_mv = cp.Problem(cp.Minimize(cp.quad_form(w, Sigma)), [cp.sum(w) == 1])
prob_mv.solve()
w_minvar = w.value

# 2. Efficient frontier: sweep target returns
frontier = []
for target in np.linspace(mu.min(), mu.max(), 40):
    constraints = [cp.sum(w) == 1, mu @ w == target]
    cp.Problem(cp.Minimize(cp.quad_form(w, Sigma)), constraints).solve()
    frontier.append((target, np.sqrt(w.value @ Sigma @ w.value)))

# 3. Constrained re-solve: long-only, 15% cap, turnover limit vs w_prev
constraints = [cp.sum(w) == 1, w >= 0, w <= 0.15,
               cp.norm1(w - w_prev) <= turnover_limit]
prob_c = cp.Problem(cp.Maximize(mu @ w - 2*cp.quad_form(w, Sigma)), constraints)
prob_c.solve()
w_constrained = w.value
```

Common Beginner Mistakes

- Treating the optimiser's output as reliable simply because the solver converged — convergence confirms the algorithm stopped, not that the problem's inputs were sound or that the answer is economically sensible.
- Using sample covariance matrices without considering estimation error, resulting in unstable portfolio weights that swing dramatically with small changes in the estimation window.
- Optimising directly for historical Sharpe ratio and inadvertently overfitting the portfolio to the specific backtest period used to estimate its inputs.
- Ignoring transaction costs, turnover, liquidity, and market-impact constraints, implicitly assuming trading is free and unlimited in size.
- Adding excessive constraints without understanding how they alter the economic objective — a long, ad hoc constraint list can leave little of the original optimisation problem intact.
- Assuming convex optimisation automatically produces an economically sensible solution, when convexity only guarantees the mathematics of the search, not the quality of the inputs.

Advanced Extensions

The following are Depth 3+ topics, introduced here only by name so you know what to look for when you reach them:

- Robust optimisation and distributionally robust optimisation — formulating and solving min-max problems over an explicit uncertainty set of possible inputs.
- Black-Litterman optimisation — combining market-implied equilibrium returns with an investor's subjective views in a statistically principled Bayesian framework.
- CVaR optimisation and risk budgeting — optimising directly against tail-risk measures rather than variance, and allocating risk budgets across strategies or desks rather than assets alone.
- Factor-neutral and sparse portfolio optimisation — constraining exposure to specific risk factors to zero, and penalising the number of non-zero positions directly.
- Stochastic and dynamic portfolio optimisation, model-predictive control, and reinforcement-learning-based optimisation — extending single-period optimisation into genuinely multi-period, adaptive decision-making (Level 5).
- Mixed-integer optimisation — handling genuinely discrete decisions (e.g., minimum lot sizes, a fixed maximum number of holdings) that a continuous QP cannot represent.

Professional Applications

- Portfolio construction and asset allocation — translating return forecasts and risk estimates into an implementable set of positions, at both the security and asset-class level.
- Risk budgeting and risk parity — allocating capital across strategies, desks, or asset classes according to a target risk contribution rather than a target capital amount.

- Statistical arbitrage and market-neutral strategies — using optimisation to size a large number of simultaneous positions while satisfying dollar-neutrality, factor-neutrality, and leverage constraints.
- Hedging and execution optimisation — determining hedge ratios and liquidity-aware execution schedules that balance market impact against the risk of an unhedged or slowly-executed position.
- Factor exposure management and capital allocation — keeping a portfolio's exposure to systematic factors within mandated limits while optimising for the residual, idiosyncratic opportunity set.

Where to Go Next

This module deliberately stopped at awareness-level for the KKT conditions, robust optimisation, and the full range of Advanced Extensions above. Depth 3+ of this same module returns to each of them quantitatively — with explicit derivations, solver implementations, and calibration exercises — and is the appropriate next stop before attempting to design or operate a production portfolio-construction system, per the prerequisite gates throughout this guide. From there, the curriculum's Execution Algorithms and Risk Metrics modules build directly on the transaction-cost and risk-budgeting foundations introduced in Part 6.